

PLAYBOOK AIDD — Do Zero ao Deploy

Proposta de estrutura do playbook operacional da obra *AI Driven Development: Do Zero ao Deploy*

Fábrica Agêntica de Publicações

Agosto 2026

Sumário

1. O que é este documento	1
2. Arquitetura proposta (visão macro)	2
3. Bloco 0 — Como usar o playbook	3
3.1 Personas atendidas	3
3.2 Contrato do Mestre de Obras (abre o playbook)	3
3.3 Convenções de notação (usadas em todos os cards)	3
4. Bloco 1 — Mapa da Obra	3
5. Bloco 2 — As 20 Missões (formato do card)	4
5.1 Tabela-mestre das 20 missões	6
5.2 Exemplo de card totalmente escrito (missão 13, como padrão de referência)	8
6. Bloco 3 — Biblioteca de Artefatos	8
7. Bloco 4 — Biblioteca de Prompts	10
8. Bloco 5 — Protocolos e Gates	10
9. Bloco 6 — Painéis e Métricas	11
10. Bloco 7 — Trilhas Alternativas	11
11. Bloco 8 — Kit do Instrutor	11
12. Anexos	12
13. Decisões editoriais recomendadas	12
14. Como produzir este playbook na Fábrica	13
15. Próximo passo sugerido	13

1. O que é este documento

Proposta de **estrutura completa** para um Playbook operacional derivado do livro *AI Driven Development: Do Zero ao Deploy* (v2, XG, 284 págs., 5 Partes, 20 capítulos, projeto-fio-condutor **Torre-DeControle**).

O livro **ensina**. O playbook **faz executar**: converte os 20 capítulos em 20 missões com entregável, verificador determinístico, critério de pronto e prompt pronto para colar no agente. Um leitor que termina o playbook não termina com anotações — termina com um repositório versionado, testado, auditado e em produção.

Dimensão	Livro (existente)	Playbook (proposto)
Unidade	Capítulo (EITA, 7 seções)	Missão (card operacional de 1 página)
Verbo dominante	Explicar, ilustrar	Executar, verificar, registrar
Prova de aprendizado	Exercício ao fim da seção <i>Aplica</i>	Gate automatizado (verificar_*.py) + evidência no repositório
Leitura	Linear, 284 páginas	Consulta por missão, ~90 páginas + anexos destacáveis
Saída	Conhecimento	TorreDeControle em produção + portfólio

Princípio de projeto: o playbook **não repete** a teoria do livro. Ele referencia (→ Cap. 7 §4) e entrega o que o livro não pode entregar em prosa: formulário, checklist, gate, cronograma e rubrica.

2. Arquitetura proposta (visão macro)

PLAYBOOK AIDD

- └ Bloco 0 – Como usar (7 págs.)
 - | Personais · 3 trilhas · contrato do Mestre de Obras · convenções de notação
- └ Bloco 1 – Mapa da Obra (4 págs.)
 - | 5 Estágios = 5 Partes do livro · linha do tempo · dependências entre missões
- └ Bloco 2 – As 20 Missões (60 págs.) ← núcleo
 - | 1 card por capítulo, formato rígido e idêntico
- └ Bloco 3 – Biblioteca de Artefatos (8 págs.)
 - | Todo arquivo que a TorreDeControle deve ter, com dono, gate e capítulo de origem
- └ Bloco 4 – Biblioteca de Prompts (10 págs.)
 - | Prompts de 5 partes, refinamento, verificação, revisor, diagnóstico, subagentes
- └ Bloco 5 – Protocolos e Gates (6 págs.)
 - | Os 9 protocolos do livro em formato de procedimento operacional
- └ Bloco 6 – Painéis e Métricas (5 págs.)
 - | Painel de testes · painel semanal de operação · orçamento de tokens · DORA
- └ Bloco 7 – Trilhas Alternativas (4 págs.)
 - | Fim de semana (16 h) · 30 dias (solo) · 8 semanas (turma/equipe)
- └ Bloco 8 – Kit do Instrutor (6 págs.)
 - | Rubrica de avaliação · banco de perguntas · erros esperados · critérios de badge
- └ Anexos (12 págs.)
 - | A. Checklist mestre (1 folha) · B. Mapa capítulo→artefato→gate
 - | C. Glossário do domínio · D. Matriz de riscos MCP · E. Registro de decisões (modelo)

Total estimado: **~120 páginas** (formato A4, mesmo template Typst do livro), ou **~55 páginas** na versão “somente cards + anexos” (edição de bancada).

3. Bloco 0 — Como usar o playbook

3.1 Personas atendidas

Persona	Origem	O que o playbook resolve
Mestre de Obras solo	Leitor iniciante do livro	Ordem de execução, gate de cada etapa, não travar
Time adotando AIDD	Squad de 3–8 devs	Padroniza AGENTS.md, permissões, revisão e CI entre pessoas
Instrutor / bootcamp	Formação técnica	Rubrica, entregas avaliáveis, banco de perguntas
Auditor / tech lead	Governança	Evidência objetiva do que foi feito (trilha, vereditos, painéis)

3.2 Contrato do Mestre de Obras (abre o playbook)

Uma página assinável com as 5 cláusulas destiladas do livro:

1. Nenhuma fatia entra sem verificador que a prove.
2. Nenhum segredo entra no repositório — `.env.example` documenta, plataforma injeta.
3. Nenhuma decisão de arquitetura fica só no chat — vai para `docs/deciso.es.md`.
4. Autonomia sobe por rampa (aprovar tudo → aprovar destrutivo → autônomo com hooks).
5. O agente propõe; a assinatura do commit é humana.

3.3 Convenções de notação (usadas em todos os cards)

Símbolo	Significado
→ Cap. N §S	Referência ao livro (capítulo, seção EITA)
ENTREGA	Arquivo que deve existir no repositório ao fim da missão
GATE	Comando/script que decide aprovado ou reprovado
DoD	Definition of Done — lista binária, sem “quase”
	Armadilha catalogada no livro (seção <i>Aplica</i>)
	Tempo estimado para iniciante

4. Bloco 1 — Mapa da Obra

Os 5 Estágios espelham as 5 Partes do livro e funcionam como *milestones* com corte seco: não se avança de estágio com gate vermelho.

Estágio	Parte do livro	Missões	Marco (o que existe ao final)	
E1 — Terreno Baldio	I. Fundamentos	1–4	Canteiro instalado, repositório inicial, primeiro prompt de engenharia entregue e commitado	6–8 h
E2 — Estrutura	II. Na Prática	5–8	Contexto arquitetado, manual do agente, especificável, esqueleto em 3 fatias	10–12 h
E3 — Instalações	III. Avançando	9–12	Skills, MCP conectado, tool própria blindada, subagentes orquestrados	12–14 h
E4 — Acabamento	IV. Profissionalizando	13–16	Governança por hooks, suíte RN1–RN7, revisão em 2 camadas, orçamento de tokens	12–14 h
E5 — Entrega das Chaves	V. Mundo Real	17–20	Pipeline com 4 gates, deploy em nuvem, observabilidade + DORA, portfólio	14–16 h

Grafo de dependência (renderizar como Mermaid no playbook, mesmo padrão visual do livro): M3 → M4 → M6 → M7 → M8 → {M9, M10, M11, M12} → M13 → M14 → M15 → M17 → M18 → M19 → M20. M5 e M16 são transversais: entram no início de E2 e são revisitados a cada estágio.

5. Bloco 2 — As 20 Missões (formato do card)

Formato **rígido e idêntico** para as 20 — é o que torna o playbook consultável.

MISSÃO N — <título curto e imperativo>

Fonte: Cap. N do livro · Estágio EX ·  <tempo>

- ① PERGUNTA-CHAVE uma frase que a missão responde
- ② PRÉ-REQUISITO missão anterior + estado esperado do repositório
- ③ ENTREGAS lista de arquivos com caminho exato
- ④ EXECUÇÃO passos numerados (referência ao §4 Técnica do capítulo)
- ⑤ PROMPT PRONTO bloco copiável, no padrão de 5 partes (Cap. 4)
- ⑥ GATE comando + saída esperada
- ⑦ DoD caixas binárias (5 a 7 itens)

⑧ ▲ ARMADILHAS

⑨ REGISTRO

3 erros mais comuns + sintoma + correção

o que anotar em docs/decisoões.md ou no painel

5.1 Tabela-mestre das 20 missões

#	Missão	Entrega principal	Gate	Estágio
1	Decidir o modo de trabalho (vibe/ agentic/AIDD)	docs/especificacao.md (corte inicial)	Matriz de decisão preenchida + 3 perguntas respondidas	E1
2	Mapear as 4 camadas do seu ambiente	docs/mapa_camadas.md executado	Checklist de demandas agnóstico de camada sem lacuna	E1
3	Erguer o canteiro	.gitignore, README.md, estrutura de pastas, commit inicial	verificar_ambiente.py → CANTEIRO PRONTO	E1
4	Escrever o primeiro prompt de engenharia	Primeira entrega real gerada pelo agente + commit	py_compile OK + entrega bate com RF3	E1
5	Arquitetar o contexto texto	docs/mapa_contexto.md, docs/estado_sessao.md, diario_decisoes.py	Sessão reaberta a partir do resumo responde no nível anterior	E2
6	Escrever o manual de bordo	AGENTS.md, CLAUDE.md	verificar_manual.py OK + agente recita as regras de segurança	E2
7	Modelar o domínio	Glossário, modelo ER, RN1- RN7, critérios de aceite do RF3	verificar_spec.py → estrutura OK	E2
8	Gerar o esqueleto em 3 fatias	app/ scaffoldado, 3 commits (função, colunas, laje)	verificar_esqueleto.py + suíte de testes verde	E2
9	Criar as skills do projeto	.claude/ skills/ adicionar- rota-api/, .../ revisar- codigo-gerado/, catálogo	verificar_skills.py + skill invocada numa rota nova	E3
10	Conectar o agente ao mundo (MCP)	Config MCP (banco local + 1 API externa, escopo mínimo)	verificar_mcp.py + teste conversacional lista tabelas	E3
11	Construir uma tool própria e blindada	app/tools/mover_tarefa.py, servidor_tools.py, test_seguranca_tools.py	Transição inválida bloqueada com 422 ponta a ponta	E3
12	Realizar o teste de integração	docs/teste_integracao.md, testes de integração	test_integracao.py	E4

5.2 Exemplo de card totalmente escrito (missão 13, como padrão de referência)

MISSÃO 13 – INSTALAR O PORTEIRO

Fonte: Cap. 13 · Estágio E4 · 🕒 90 min

① PERGUNTA-CHAVE

Que autonomia o agente tem hoje, e o que o impede de passar do limite?

② PRÉ-REQUISITO

M8 concluída (esqueleto), M14 pode rodar em paralelo. Repositório com git limpo.

③ ENTREGAS

docs/mapa_permissoes.md (livres · com aprovação · proibidos · arquivos vedados · escopos MCP)
.hooks/bloquear_push_forcado.sh (executável)
configuração de hooks no harness (bloqueio + registro)

④ EXECUÇÃO → Cap. 13 §4, passos 1 a 4

1. Preencher o mapa de permissões nas 5 seções.
2. Registrar os hooks no harness (evento pré-comando e pós-teste).
3. Tornar o script de bloqueio executável e testá-lo isoladamente.
4. Rodar verificar_governanca.py.

⑤ PROMPT PRONTO

Papel e contexto – Você é o agente do projeto TorreDeControle; o mapa de permissões em docs/mapa_permissoes.md é a única fonte de verdade.
Tarefa específica – Proponha o conteúdo do hook que bloqueia comandos da seção "proibidos", lendo-a do arquivo.
Restrições – Não altere o mapa. Não use rede. Shell POSIX.
Formato de saída – 1 arquivo .sh + 3 linhas explicando o gatilho.
Critérios de aceite – `git push --force` retorna código ≠ 0 e mensagem clara.

⑥ GATE

python verificar_governanca.py → "GOVERNANÇA OK"
git push --force (em branch de teste) → bloqueado pelo hook

⑦ DoD

- [] As 5 seções do mapa preenchidas, sem "etc."
- [] Hook bloqueia comando proibido e o registra na trilha
- [] Trilha de auditoria contém a tentativa bloqueada
- [] Nível de autonomia atual declarado no README (rampa 1, 2 ou 3)
- [] Commit "gov: mapa de permissões e hooks" no log

⑧ ⚠ ARMADILHAS

Hook silencioso (bloqueia sem mensagem) → sintoma: dev acha que é bug do git.
Permissão ampla demais ("bash: *") → sintoma: nada nunca é barrado.
Mapa escrito depois do incidente → sintoma: governança vira relatório, não controle.

⑨ REGISTRO

docs/decisoes.md: nível de autonomia escolhido + por quê + data de revisão.

6. Bloco 3 — Biblioteca de Artefatos

Tabela única com **tudo** que a TorreDeControle acumula. Serve de checklist de auditoria final e de índice reverso (do arquivo para o capítulo).

Artefato	Tipo	Missão	Gate que o cobre
README.md	Doc	3	verificar_ambiente.py
.gitignore	Config	3	verificar_ambiente.py
docs/ especificacao.md	Spec	1, 7	verificar_spec.py
docs/ mapa_camadas.md	Doc	2	Checklist manual
docs/ mapa_contexto.md	Doc	5	Revisão de sessão
docs/ estado_sessao.md	Doc vivo	5	Reabertura de sessão
docs/decisoões.md	Registro	5→20	diario_decisoões.py
docs/memoria.md	Registro	16	Revisão trimestral
docs/ mapa_permissões.md	Governança	13	verificar_governanca.py
docs/vereditos.md	Registro	15	registrar_veredito.py
AGENTS.md / CLAUDE.md	Manual	6, 16	verificar_manual.py
.claude/skills/**	Skill	9	verificar_skills.py
Config MCP do har- ness	Config	10	verificar_mcp.py
app/tools/ mover_tarefa.py	Código	11	test_seguranca_tools.py
app/tools/ servidor_tools.py	Código	11	Teste ponta a ponta
Definições de suba- gentes	Prompt	12	verificar_subagentes.py
tests/test_rn*.py	Teste	14	verificar_cobertura_testes.py
ci_sintaxe.sh	Gate	14	Hook pré-commit
auditar_repositorio.py	Gate	15	Execução limpa
orcamento_tokens.py	Gate	16	Registro semanal
requirements.lock.txt + Dockerfile	Build	17	testar_pipeline.py
Pipeline CI (YAML)	CI/CD	17	4 gates verdes
app/config.py + .env.example	Config	18	smoke_test_producao.py
scripts/migrar.py	Migração	18	Migração idempotente
logging_config.py, metricas.py, health.py	Observabilidade	19	/health 200
relatorio_dora.py	Métrica	19	Painel semanal
gerar_portfolio.py + doc da jornada	Portfólio	20	Pitch de 30 s

Regra do bloco: artefato sem gate correspondente é candidato a corte — ou ganha gate, ou sai do playbook.

7. Bloco 4 — Biblioteca de Prompts

Sete famílias, todas já presentes no livro, reunidas em páginas destacáveis:

#	Família	Origem	Uso
P1	Prompt completo de 5 partes	Cap. 4 §4	Toda entrega nova
P2	Prompt de refinamento (iteração)	Cap. 4 §4	Após veredito com ajustes
P3	Prompt de verificação (questionar antes de codar)	Cap. 4 §4	Antes de fatia arriscada
P4	Prompt de resumo de contexto	Cap. 5 §4	Encerramento de sessão
P5	Prompt de geração de testes	Cap. 14 §4	RN sem cobertura
P6	Prompt do revisor agêntico	Cap. 15 §4	Camada 2 da revisão
P7	Prompt de diagnóstico assistido	Cap. 19 §4	Anomalia em produção

Cada prompt entra em página única com: quando usar, gabarito preenchido para a TorreDeControle, gabarito em branco, e o erro típico de preenchimento.

8. Bloco 5 — Protocolos e Gates

Os protocolos do livro convertidos em POP (procedimento operacional padrão) de meia página, com entrada, passos, saída e critério de parada:

1. **Verificação de camadas** (Cap. 2) — triagem tool → harness → LLM.
2. **Higiene de sessão / três tempos** (Cap. 5) — abertura, meio, encerramento.
3. **Manutenção do manual** (Cap. 6) — gatilho de revisão por tamanho e por tempo.
4. **Revisão de fatia** (Cap. 8) — checklist de aceitação do código gerado.
5. **Criação de skill** (Cap. 9) — promoção de procedimento repetido a skill.
6. **Conexão segura MCP** (Cap. 10) — origem, escopo, postura, auditoria.
7. **Promoção de autonomia** (Cap. 13) — rampa 1→2→3 com critérios objetivos.
8. **TDD com agente** (Cap. 14) — teste primeiro, agente depois.
9. **Rollback e incidente** (Cap. 13 e 18) — o que fazer quando a produção cai.

Gate consolidado (o “portão único”): uma página com a sequência que deve rodar verde antes de qualquer merge — `ci_sintaxe.sh` → `auditar_repositorio.py` → `revisor_agêntico` → `verificar_cobertura_testes.py`.

9. Bloco 6 — Painéis e Métricas

Quatro painéis em formato de formulário preenchível (uma página cada, também em .md para copiar ao repositório):

Painel	Fonte	Cadência	Decisão que ele suporta
Painel de Testes	Cap. 14	A cada fatia	O que ainda não tem prova?
Painel Semanal de Operação	Cap. 19	Semanal	O que a produção está dizendo?
Orçamento de Tokens	Cap. 16	Semanal	A obra cabe no orçamento?
Métricas DORA	Cap. 19	Mensal	O método está acelerando ou só correndo?

Cada painel traz **faixas de referência** (“verde/amarelo/vermelho”) para o iniciante saber interpretar o número — o que o livro deixa como narrativa, o playbook fixa como limiar.

10. Bloco 7 — Trilhas Alternativas

Trilha	Público	Recorte	Duração
Sprint de fim de semana	Quem quer o gostinho completo	M3, M4, M6, M7, M8, M17, M18 (spec mínima, deploy simples)	16 h
30 dias solo	Leitor padrão	20 missões, ~1 h/dia útil + 2 sessões longas de fim de semana	30 dias
8 semanas em equipe	Squad adotando AIDD	2 estágios por par de semanas, com revisão cruzada entre pessoas	8 semanas
Turma / bootcamp	Instrutor	20 missões = 20 aulas-laboratório, entrega avaliada por rubrica	1 semestre

Cada trilha tem sua própria folha de rota (missões, cortes permitidos, o que **não** pode ser cortado — governança, testes e gates nunca saem).

11. Bloco 8 — Kit do Instrutor

- **Rubrica de avaliação** por missão: 0 (não entregue) · 1 (entregue sem gate) · 2 (gate verde) · 3 (gate verde + registro de decisão + armadilha evitada documentada).
- **Banco de perguntas** (5 por capítulo, resposta com referência à seção do livro).
- **Erros esperados por missão** — colhidos das seções *Armadilhas Comuns* dos 20 capítulos; o instrutor sabe onde a turma vai travar antes de travar.
- **Badges de progresso**: Fundação · Estrutura · Instalações · Acabamento · Chaves. Cada badge exige o gate do estágio, não a leitura.

- **Projeto alternativo** para quem não quer a TorreDeControle: contrato mínimo do domínio (7 regras de negócio, 1 entidade central, 1 transição de status) para adaptar as missões.

12. Anexos

Anexo	Conteúdo	Formato
A	Checklist mestre em 1 folha (as 20 missões + DoD resumido)	Imprimível A4
B	Mapa capítulo → artefato → gate (índice reverso)	Tabela
C	Glossário do domínio TorreDeControle + glossário AIDD	Lista
D	Matriz de riscos de servidores MCP (Cap. 10)	Tabela
E	Modelos: docs/decisoões.md, docs/memoria.md, painel semanal	Blocos copiáveis
F	Índice de referências do livro por tema (as 20+ refs./capítulo)	Lista ABNT

13. Decisões editoriais recomendadas

Questão	Recomendação	Motivo
Metáfora do canteiro	Manter , mas em dose menor	O playbook é bancada, não narrativa; a metáfora fica nos títulos e no vocabulário de estágio
Repetir teoria do livro	Não	Referência → Cap. N §5 mantém o playbook fino e o livro necessário
Nível declarado	PARA INICIANTES , mesmo badge do livro	Coerência de série (gate validar-capa-nivel.py)
Formato de código	Só o que é copiável e executável ; explicação vai para o livro	Playbook de bancada
Diagramas	6 no total (grafo de missões, rampa de autonomia, 2 camadas de revisão, pipeline, loop de iteração, ciclo de vida de skill)	Mermaid, mesmo pipeline renderizar-diagramas.py
Referências ABNT	Herdadas do livro no Anexo F; sem exigência de 20/capítulo	Playbook é derivado, não obra primária
Versionamento	PLAYBOOK-aidd v1.0 atrelado ao livro v2	Playbook acompanha a versão da obra-mãe

14. Como produzir este playbook na Fábrica

O playbook é um **derivado** da obra-mãe — mesmo caminho já usado para artigos e e-books:

	Etapas	Comando / agente	Saída
1.	Fatiar a obra-mãe	<code>scripts/fatiar-obra.py -- slug ai-driven-development-do-zero-ao-deploy-v2 --alvo playbook</code>	Recortes por capítulo (seções 4 <i>Técnica</i> e 5 <i>Aplica</i>)
2.	Indexar para RAG	<code>scripts/indexar-dossie.py --indexar</code>	Dossiê consultável (sem nova pesquisa web)
3.	Gerar os 20 cards	Subagentes em lote (<code>pool-capitulos.py --plano --lote 4</code>), 1 card por capítulo, formato fixo do §5	20 arquivos <code>missao-NN.md</code>
4.	Blocos 0, 1, 3–8 e anexos	Redação direta a partir das tabelas deste documento	<code>playbook_final.md</code>
5.	Auditoria	<code>auditar-obra.py --tipo playbook + validar-codigo.py</code> (todo bloco de código deve compilar)	Parecer CONFORME
6.	Diagramas	<code>renderizar-diagramas.py --validar</code>	6 PNG
7.	Capa	Padrão Editora Agêntica 2D plano + badge de nível (<code>validar-capa-nivel.py</code>)	<code>capa.png</code>
8.	PDF	<code>compilar-para-pdf.py <slug-playbook> --paginas-exatas</code> (Pandoc → .typ → Typst)	<code>PLAYBOOK-aidd.pdf</code>
9.	Distribuição	Copiar para <code>distribuicao/</code> junto ao livro, artigos e e-books	Pacote autocontido

Regra de qualidade específica do playbook (proposta de novo gate): todo card deve ter gate executável. Sugestão de verificação determinística — `validar-playbook.py`:

- R-PBK-1: todo card tem as 9 seções (① a ⑨).
- R-PBK-2: todo card cita ao menos 1 arquivo de entrega com caminho.
- R-PBK-3: todo card tem comando de gate executável (linha iniciada por `python`, `pytest`, `bash` ou `git`).
- R-PBK-4: DoD com 5 a 7 itens binários.
- R-PBK-5: nenhuma seção do card excede 25 linhas (playbook é de bancada, não é livro).

15. Próximo passo sugerido

1. Aprovar (ou ajustar) a arquitetura dos 9 blocos do §2.
2. Validar o **card-padrão** do §5.2 — é ele que se replica 20 vezes.
3. Rodar as etapas 1–3 do §14 para gerar os 20 cards em lote e revisar 3 amostras (uma de estágio inicial, uma de meio, uma de deploy).

4. Fechar blocos 0/1/3–8 e compilar.

Estimativa de produção pela esteira autônoma: **1 ciclo**, mesma ordem de grandeza dos 5 e-books derivados (que preservaram 78–81% do conteúdo original).